



**FACULTAD DE INGENIERÍA
ESCUELA DE COMPUTACIÓN**

Análisis y Diseño de Sistemas y Base de Datos (ASB404 G01T)

**Análisis de vulnerabilidades y metodologías de
seguridad en aplicaciones web: estrategias para su
detección y mitigación**

Artículo analizado: *Evaluation of methodologies in web application security: A systematic literature review* (LACCEI, 2025)

Docente:

[Nombre del docente]

Integrante:

Genesis Scarlett Ramirez Torres

Carnet: RT262263

Fecha de entrega:

[Fecha]

ÍNDICE

- 1. INTRODUCCIÓN5
 - 1.1 Contexto de las aplicaciones web5
 - 1.2 Problemas de seguridad en aplicaciones web5
 - 1.3 Importancia de proteger las aplicaciones web5
 - 1.4 Justificación6
 - 1.5 Objetivo general6
 - 1.6 Objetivos específicos6
 - 1.7 Alcance6
- 2. DATOS DEL ARTÍCULO CIENTÍFICO6
 - 2.1 Título6
 - 2.2 Autores7
 - 2.3 Año7
 - 2.4 Publicación7
 - 2.5 DOI o enlace7
 - 2.6 Tema7
 - 2.7 Tipo de investigación7
 - 2.8 ¿Por qué escogimos este artículo?7
- 3. RESUMEN DEL ARTÍCULO8
 - 3.1 Objetivo8
 - 3.2 Problema8
 - 3.3 Cómo realizaron la investigación8
 - 3.4 Resultados8
 - 3.5 Conclusiones8
- 4. SEGURIDAD EN APLICACIONES WEB9
 - 4.1 Aplicaciones web9
 - 4.2 Seguridad9
 - 4.3 Vulnerabilidad9
 - 4.4 Riesgo10
 - 4.5 Consecuencias10
- 5. PRINCIPALES VULNERABILIDADES10

- 5.1 Inyección SQL 10
 - 5.1.1 ¿Qué es? 10
 - 5.1.2 ¿Cómo ocurre? 11
 - 5.1.3 ¿Qué daños puede causar? 11
 - 5.1.4 ¿Cómo puede prevenirse? 11
- 5.2 Cross-Site Scripting (XSS) 11
 - 5.2.1 ¿Qué es? 11
 - 5.2.2 ¿Cómo ocurre? 11
 - 5.2.3 ¿Qué daños puede causar? 11
 - 5.2.4 ¿Cómo puede prevenirse? 11
- 5.3 Cross-Site Request Forgery (CSRF) 12
 - 5.3.1 ¿Qué es? 12
 - 5.3.2 ¿Cómo ocurre? 12
 - 5.3.3 ¿Qué daños puede causar? 12
 - 5.3.4 ¿Cómo puede prevenirse? 12
- 5.4 Problemas de autenticación 12
 - 5.4.1 ¿Qué significa? 12
 - 5.4.2 ¿Qué problemas pueden presentarse? 12
 - 5.4.3 ¿Cómo pueden prevenirse? 13
- 5.5 Problemas de autorización 13
 - 5.5.1 ¿Qué significa? 13
 - 5.5.2 ¿Qué problemas pueden presentarse? 13
 - 5.5.3 ¿Cómo pueden prevenirse? 13
- 6. ¿CÓMO SE PUEDEN DETECTAR? 13**
 - 6.1 Revisión del código 14
 - 6.2 Pruebas de la aplicación 14
 - 6.3 Herramientas automáticas 14
 - 6.4 Revisión manual 14
 - 6.5 Comparación 15
 - 6.6 Combinación de métodos 15
- 7. RESULTADOS Y ANÁLISIS 15**
 - 7.1 Principales resultados 15
 - 7.2 Vulnerabilidades más importantes 15

7.3 Comparación de métodos 15

7.4 Ventajas 16

7.5 Limitaciones 16

7.6 Nuestra opinión 16

7.7 ¿Estamos de acuerdo? 16

8. APLICACIÓN PRÁCTICA 16

8.1 Mejoras propuestas 16

8.2 Medidas de seguridad 17

8.3 Aplicación de lo aprendido 17

8.4 Proyecto de Cátedra 17

8.4.1 Nombre 17

8.4.2 Problemas de seguridad 17

8.4.3 Medidas 17

8.4.4 Revisión del sistema 18

8.4.5 Beneficios 18

9. CONCLUSIÓN PERSONAL 18

9.1 Qué aprendimos 18

9.2 Qué aprendimos sobre seguridad 19

9.3 Qué aprendimos sobre vulnerabilidades 19

9.4 Qué aprendimos sobre detección 19

9.5 Cambio de perspectiva 19

9.6 Reflexión final 19

10. REFERENCIAS BIBLIOGRÁFICAS 19

10.1 Artículo principal 19

10.2 Otras fuentes 20

10.3 Documentación 20

11. ANEXOS 20

11.1 Diagramas 20

11.2 Tablas 20

11.3 Material adicional 21

1. INTRODUCCIÓN

1.1 Contexto de las aplicaciones web

Hoy en día casi todo lo que hacemos en internet ocurre dentro de una aplicación web: comprar comida, pagar un servicio, revisar las notas de la universidad o hablar con el banco. Una aplicación web es un programa al que se entra desde el navegador, sin instalar nada, y que guarda y procesa información en un servidor. Por esa comodidad, las empresas, las instituciones educativas y los pequeños negocios dependen cada vez más de ellas.

A diferencia de los programas tradicionales, que se instalan en cada computadora, una aplicación web vive en un servidor y se usa a través de internet. Esto facilita las actualizaciones y permite que miles de personas la usen al mismo tiempo desde cualquier dispositivo. Ejemplos cotidianos son la banca en línea, las plataformas educativas, las tiendas virtuales y los sistemas de gestión de pequeños negocios como una cafetería.

Esa misma ventaja es también su punto débil: como cualquiera con internet puede llegar a la aplicación, cualquiera puede también intentar atacarla. Por eso la seguridad se vuelve parte esencial del desarrollo y no un detalle opcional.

1.2 Problemas de seguridad en aplicaciones web

Una aplicación web es construida por personas y, por lo tanto, puede tener errores. Algunos errores son simples descuidos al programar, como confiar en todo lo que escribe el usuario en un formulario. Otros están en la forma de controlar quién puede entrar y qué puede hacer. Cuando un atacante encuentra uno de estos errores, puede aprovecharlo para ver información que no le corresponde, modificar datos o actuar en nombre de otra persona.

Los problemas de seguridad rara vez se deben a algo extraordinario. En la mayoría de los casos son errores comunes y repetidos: no revisar lo que escribe el usuario, guardar contraseñas sin protegerlas, o permitir que un usuario común llegue a opciones de administrador. Como estos errores se repiten en muchos sistemas, existen listas públicas, como la OWASP Top 10, que los recopilan para que los equipos aprendan a evitarlos.

1.3 Importancia de proteger las aplicaciones web

Las aplicaciones web guardan datos personales, contraseñas, pedidos y pagos. Si no se protegen, las consecuencias llegan a las personas (robo de datos), a la organización (pérdida de dinero y de confianza) y al propio sistema (interrupciones o información alterada). Proteger una aplicación no es un lujo del final del proyecto: es parte de hacerla bien desde el principio.

Para una institución educativa o un negocio, una filtración de datos implica perder la confianza de sus usuarios, posibles sanciones legales y costos de recuperación. Para un desarrollador, implica la responsabilidad profesional de haber entregado un sistema que no

cuidó a las personas que lo usan. Proteger la aplicación desde el diseño siempre es más barato que repararla después de un ataque.

1.4 Justificación

Escogimos este tema porque, como estudiantes de desarrollo de software, estamos construyendo sistemas que manejan usuarios, sesiones y datos reales. Entender qué vulnerabilidades existen y cómo se revisan nos permite programar mejor y evaluar con criterio nuestro propio Proyecto de Cátedra. Además, el artículo elegido reúne el trabajo de muchos investigadores, lo que nos da una visión más amplia que la de un estudio aislado.

1.5 Objetivo general

Analizar el artículo científico seleccionado sobre metodologías de seguridad en aplicaciones web, para comprender las principales vulnerabilidades, las formas de detectarlas y las medidas para reducirlas, y relacionar lo aprendido con nuestro proyecto.

1.6 Objetivos específicos

Identificar las vulnerabilidades más comunes en aplicaciones web.

Explicar cómo pueden detectarse.

Analizar los resultados del artículo.

Explicar formas de reducir los riesgos.

Relacionar lo aprendido con nuestro proyecto.

1.7 Alcance

El trabajo se limita a la seguridad de aplicaciones web. Estudiamos cinco temas: inyección SQL, Cross-Site Scripting (XSS), Cross-Site Request Forgery (CSRF), autenticación y autorización. También estudiamos los métodos para detectarlos. No cubrimos seguridad de redes, de servidores ni de dispositivos móviles, ni realizamos pruebas de ataque reales.

El análisis se centra en el artículo seleccionado y en los conceptos mínimos necesarios para entenderlo. La parte práctica se aplica a un sistema real: el Proyecto de Cátedra Café Don Bosco.

2. DATOS DEL ARTÍCULO CIENTÍFICO

2.1 Título

Evaluation of methodologies in web application security: A systematic literature review (en español: Evaluación de metodologías en la seguridad de aplicaciones web: una revisión sistemática de la literatura).

2.2 Autores

Quesquen Farroñay, J. A., Bances Quevedo, J. J., Garces Rosendo, E. J. y Osoreo Granda, O. E. Los cuatro autores pertenecen a la Universidad Tecnológica del Perú (Lima).

2.3 Año

2025.

2.4 Publicación

Memorias (proceedings) de la conferencia LACCEI 2025, realizada en México del 16 al 18 de julio de 2025 (Latin American and Caribbean Consortium of Engineering Institutions).

2.5 DOI o enlace

DOI: 10.18687/LACCEI2025.1.1.526. Enlace:
<https://doi.org/10.18687/LACCEI2025.1.1.526>

2.6 Tema

Comparación y evaluación de las metodologías que se usan para revisar la seguridad de las aplicaciones web, con atención a ataques como la inyección SQL y el Cross-Site Scripting.

Este tema pertenece al área de la ingeniería de software segura. Su importancia radica en que no basta con conocer los ataques: un equipo también necesita saber con qué método revisar su sistema y cómo interpretar los resultados de esa revisión.

2.7 Tipo de investigación

Es una revisión sistemática de la literatura. Esto quiere decir que los autores no hicieron un experimento propio, sino que reunieron investigaciones anteriores sobre el tema, las filtraron con reglas claras y analizaron en conjunto lo que ellas encontraron.

2.8 ¿Por qué escogimos este artículo?

Lo escogimos porque es reciente (2025), está publicado en una conferencia de ingeniería de América Latina, y su tema encaja directamente con nuestro curso: no solo habla de qué ataques existen, sino de cómo evaluar y comparar las formas de revisar la seguridad. Al ser una revisión de muchos estudios, nos da una visión general que podemos aplicar a nuestro proyecto.

3. RESUMEN DEL ARTÍCULO

3.1 Objetivo

El artículo busca evaluar las metodologías disponibles para revisar la seguridad de las aplicaciones web y entender cuáles se usan, cómo se comparan y qué tan bien funcionan.

3.2 Problema

Los ciberataques contra aplicaciones web son frecuentes y existen muchas metodologías y herramientas para detectarlos, pero según los autores no hay un acuerdo sobre cuál es la mejor ni criterios comunes para elegir entre ellas. Esto dificulta la decisión de quienes deben proteger un sistema.

En otras palabras, el problema no es la falta de métodos, sino la abundancia de ellos sin una guía clara. Un equipo de desarrollo que quiere revisar su aplicación se encuentra con herramientas automáticas, técnicas manuales, revisiones de código y pruebas en ejecución, y no sabe con certeza cuál conviene usar ni cómo comparar sus resultados.

3.3 Cómo realizaron la investigación

Los autores siguieron un proceso ordenado, explicado aquí de forma sencilla:

Buscaron investigaciones anteriores en bases de datos científicas (entre ellas Scopus).

Revisaron los resultados encontrados (según el resumen consultado, más de mil documentos al inicio).

Aplicaron criterios para escoger cuáles sí servían y cuáles no.

Finalmente analizaron 53 estudios.

Para ordenar este proceso, los autores usaron métodos propios de las revisiones sistemáticas; en este trabajo no los desarrollamos porque basta con entender la idea general.

3.4 Resultados

Lo que se ha podido confirmar es que el artículo gira en torno a ataques de entrada de datos (inyección SQL y XSS) y a la comparación de metodologías de evaluación. El detalle de los resultados debe tomarse del artículo.

3.5 Conclusiones

Los autores concluyen que hacen falta criterios estandarizados para elegir las metodologías de evaluación de seguridad, y que no existe consenso sobre cuál es el mejor enfoque, lo que representa un reto importante para los profesionales de la seguridad.

Dicho de manera sencilla: no existe todavía una receta única y aceptada por todos para evaluar la seguridad de una aplicación web, por lo que cada equipo debe elegir y combinar métodos con criterio propio.

4. SEGURIDAD EN APLICACIONES WEB

4.1 Aplicaciones web

Una aplicación web funciona así: la persona usa el navegador, el navegador envía una petición a la aplicación que está en un servidor, y la aplicación consulta o guarda datos en una base de datos para devolver una respuesta. La Figura 1 muestra este recorrido.

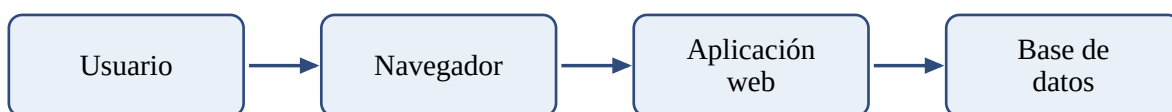


Figura 1. Funcionamiento general de una aplicación web.

En este recorrido hay varios puntos donde puede fallar la seguridad: lo que el usuario escribe en el navegador, la comunicación con el servidor, la lógica de la aplicación y la base de datos. Un atacante busca el punto más débil de toda la cadena.

4.2 Seguridad

La seguridad de una aplicación web es el conjunto de medidas que impiden que personas no autorizadas vean, cambien o destruyan la información, y que aseguran que el sistema siga funcionando para quienes sí deben usarlo. Se suele resumir en tres ideas: confidencialidad (solo ve quien debe ver), integridad (los datos no se alteran) y disponibilidad (el sistema está disponible cuando se necesita).

Por ejemplo, en una tienda en línea la confidencialidad protege los datos de los clientes, la integridad evita que alguien cambie el precio de un producto, y la disponibilidad garantiza que la tienda esté abierta cuando los clientes quieren comprar.

4.3 Vulnerabilidad

Una vulnerabilidad es una debilidad o error en la aplicación que podría ser aprovechado por un atacante. Por ejemplo, un formulario que acepta cualquier texto sin revisarlo.

Las vulnerabilidades pueden estar en el código escrito por el equipo, en las librerías de terceros o en la configuración del servidor. Algunas pasan años sin ser descubiertas; otras son encontradas por atacantes antes que por los propios desarrolladores, y de ahí la importancia de buscarlas activamente.

4.4 Riesgo

El riesgo es la posibilidad de que una vulnerabilidad sea aprovechada, combinada con lo grave que sería el daño. Una vulnerabilidad en una página sin datos importantes tiene menos riesgo que la misma vulnerabilidad en una página de pagos.

Para decidir qué corregir primero, los equipos suelen clasificar los riesgos por nivel (bajo, medio, alto o crítico), considerando qué tan fácil es explotar la vulnerabilidad y qué tan grave sería el daño. Esto permite usar bien el tiempo limitado de un proyecto.

4.5 Consecuencias

Cuando una vulnerabilidad es explotada, el proceso suele ser el de la Figura 2. Las consecuencias pueden ser:

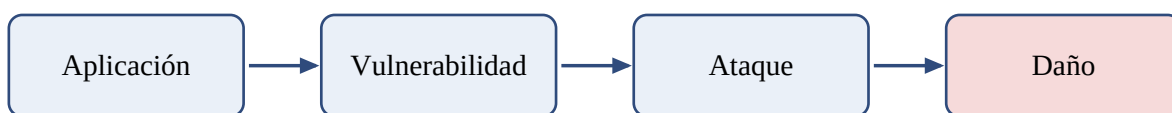


Figura 2. Del error en la aplicación al daño.

Robo o filtración de datos personales y contraseñas.

Modificación o borrado de información (por ejemplo, precios o pedidos).

Acciones no autorizadas hechas a nombre de un usuario legítimo.

Pérdida económica y daño a la reputación de la institución o negocio.

Interrupción del servicio.

5. PRINCIPALES VULNERABILIDADES

Las vulnerabilidades de este capítulo están entre las más conocidas y aparecen de forma recurrente en la lista OWASP Top 10, un listado de referencia de los riesgos más comunes en aplicaciones web (OWASP Foundation, 2021).

5.1 Inyección SQL

Ejemplo sencillo: si un inicio de sesión arma la consulta pegando el usuario y la contraseña dentro del texto SQL, un atacante puede escribir una condición que siempre sea verdadera y entrar sin conocer la contraseña. La solución es separar el código de la consulta de los datos, enviándolos como parámetros.

5.1.1 ¿Qué es?

Es un ataque en el que el atacante escribe instrucciones de base de datos (SQL) dentro de un campo que debería recibir solo texto común, como un usuario o un buscador.

5.1.2 ¿Cómo ocurre?

Ocurre cuando la aplicación arma la consulta a la base de datos pegando directamente lo que escribió el usuario. Si el usuario escribe algo como una condición siempre verdadera, la base de datos la ejecuta como parte de la consulta y puede devolver más datos de los previstos o saltarse un inicio de sesión.

5.1.3 ¿Qué daños puede causar?

Lectura de toda la base de datos, acceso sin contraseña, modificación o eliminación de registros y, en casos graves, control del servidor de base de datos.

5.1.4 ¿Cómo puede prevenirse?

Usar consultas parametrizadas (por ejemplo, PreparedStatement en Java) en lugar de concatenar texto.

Validar el tipo, el largo y el formato de cada dato recibido.

Dar a la aplicación una cuenta de base de datos con los permisos mínimos necesarios.

No mostrar al usuario los mensajes de error técnicos de la base de datos.

5.2 Cross-Site Scripting (XSS)

Existen varios tipos: el XSS almacenado, donde el código malicioso queda guardado en la base de datos y afecta a todos los que ven esa página; el reflejado, donde viene en un enlace; y el basado en el DOM, que ocurre en el propio navegador.

5.2.1 ¿Qué es?

Es un ataque en el que el atacante logra que la página web muestre código JavaScript malicioso que luego se ejecuta en el navegador de otras personas.

5.2.2 ¿Cómo ocurre?

Ocurre cuando la aplicación toma un texto que escribió alguien (un comentario, un nombre, un parámetro de la dirección) y lo muestra en la página sin convertir los caracteres especiales. El navegador interpreta ese texto como código en vez de como simple texto.

5.2.3 ¿Qué daños puede causar?

Robo de la sesión de la víctima, suplantación de identidad, redirección a páginas falsas y alteración de lo que la persona ve en pantalla.

5.2.4 ¿Cómo puede prevenirse?

Escapar (codificar) todo dato antes de mostrarlo en una página HTML.

Validar y limpiar la entrada del usuario.

Marcar la cookie de sesión como HttpOnly para que el código de la página no pueda leerla.

Usar una política de seguridad de contenido (Content-Security-Policy).

5.3 Cross-Site Request Forgery (CSRF)

A diferencia del XSS, el CSRF no roba información: abusa de la confianza que la aplicación tiene en el navegador de la víctima. Por eso las acciones que cambian datos deben exigir algo que un sitio externo no pueda adivinar, como un token único por sesión.

5.3.1 ¿Qué es?

Es un ataque que engaña al navegador de una persona que ya inició sesión para que envíe una petición no deseada a la aplicación, sin que la persona lo sepa.

5.3.2 ¿Cómo ocurre?

El navegador envía automáticamente la cookie de sesión en cada petición al sitio. Si la víctima visita una página maliciosa mientras tiene la sesión abierta, esa página puede hacer que el navegador envíe, por ejemplo, una orden de cambiar el correo, y la aplicación la acepta porque trae la cookie correcta.

5.3.3 ¿Qué daños puede causar?

Cambios de datos, compras o transferencias y modificaciones de cuenta hechas a nombre de la víctima.

5.3.4 ¿Cómo puede prevenirse?

Incluir un token antiCSRF único en cada formulario y verificarlo en el servidor.

Configurar la cookie de sesión con el atributo SameSite (Strict o Lax).

Usar métodos POST para acciones que cambian datos, nunca GET.

Pedir confirmación o contraseña para acciones sensibles.

5.4 Problemas de autenticación

La autenticación es la puerta de entrada del sistema. Si falla, todas las demás protecciones quedan sin valor, porque el atacante entra como si fuera un usuario legítimo. Por eso se recomienda que sea una de las partes más revisadas del proyecto.

5.4.1 ¿Qué significa?

Autenticación es comprobar que una persona es quien dice ser, normalmente con usuario y contraseña.

5.4.2 ¿Qué problemas pueden presentarse?

Contraseñas débiles o guardadas sin cifrar.

Intentos ilimitados de inicio de sesión, que permiten adivinar contraseñas por fuerza bruta.

Sesiones que no caducan o identificadores de sesión predecibles.

Mensajes de error que revelan si un usuario existe.

5.4.3 ¿Cómo pueden prevenirse?

Guardar las contraseñas con un algoritmo de hash seguro y con sal (por ejemplo, bcrypt), nunca en texto plano.

Limitar los intentos de inicio de sesión y bloquear temporalmente.

Renovar el identificador de sesión al iniciar sesión y hacer caducar las sesiones inactivas.

Exigir contraseñas de longitud razonable y, cuando sea posible, doble factor de autenticación (Grassi et al., 2017).

5.5 Problemas de autorización

Es común confundir autenticación con autorización: la primera responde a la pregunta de quién eres, y la segunda a qué puedes hacer. Un sistema puede identificar bien a un cliente y aun así permitirle ver los pedidos de otros por no verificar los permisos.

5.5.1 ¿Qué significa?

Autorización es decidir qué puede hacer cada usuario una vez que ya se identificó. Un cliente no debería poder usar las pantallas de un administrador.

5.5.2 ¿Qué problemas pueden presentarse?

Páginas de administración accesibles con solo conocer la dirección.

Poder ver o modificar datos de otro usuario cambiando un número en la dirección (por ejemplo, el id de un pedido).

Verificar los permisos solo en el menú visible y no en el servidor.

5.5.3 ¿Cómo pueden prevenirse?

Verificar el rol y la propiedad del recurso en el servidor en cada petición.

Aplicar el principio de mínimo privilegio: cada rol recibe solo lo que necesita.

Usar identificadores difíciles de adivinar para recursos públicos y comprobar siempre a quién pertenece cada dato.

6. ¿CÓMO SE PUEDEN DETECTAR?

Encontrar las vulnerabilidades antes que el atacante es el centro del artículo, que compara distintas metodologías de evaluación. La Figura 3 resume las dos grandes vías.

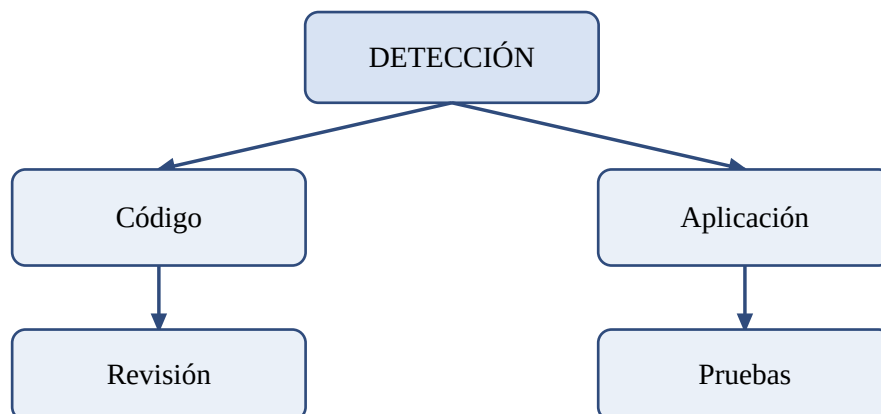


Figura 3. Formas de detectar problemas de seguridad.

6.1 Revisión del código

Consiste en leer el código fuente buscando errores de seguridad sin ejecutar la aplicación, por ejemplo consultas SQL armadas con texto o datos que se muestran sin escapar. Permite encontrar fallos desde temprano, cuando corregirlos cuesta menos.

6.2 Pruebas de la aplicación

Consiste en probar la aplicación mientras está funcionando, enviándole entradas inesperadas y observando cómo responde. Ayuda a ver el comportamiento real, incluidos problemas de configuración que el código por sí solo no muestra. Debe hacerse en un ambiente de pruebas y con autorización.

6.3 Herramientas automáticas

Son programas que revisan el código o prueban la aplicación de forma automática. Entre las herramientas de escaneo conocidas están OWASP ZAP y Burp Suite. Su ventaja es la velocidad y que pueden repetirse con frecuencia; su desventaja es que pueden pasar por alto algunos problemas o reportar falsos positivos.

Existen herramientas de análisis estático (SAST), que revisan el código sin ejecutarlo, y de análisis dinámico (DAST), que prueban la aplicación en funcionamiento. Ambas sirven como primera línea de revisión, pero sus resultados deben ser interpretados por una persona para descartar falsas alarmas.

6.4 Revisión manual

Una persona con conocimientos de seguridad revisa el sistema. Entiende la lógica del negocio (por ejemplo, que un cliente no deba ver pedidos ajenos), algo que las herramientas automáticas difícilmente detectan. Su desventaja es que requiere más tiempo.

Un ejemplo típico es la revisión de permisos: una herramienta automática no sabe que el cliente A no debería ver el pedido del cliente B, pero una persona que conoce el negocio puede comprobarlo en pocos minutos.

6.5 Comparación

Método	Ventaja	Desventaja
Automático	Más rápido	Puede no detectar algunos problemas
Manual	Más detallado	Requiere más tiempo
Código	Permite encontrar errores temprano	No muestra todo lo que ocurre al ejecutar
Aplicación funcionando	Permite observar el comportamiento	Se necesita un ambiente de pruebas

Tabla 1. Comparación entre métodos de detección.

6.6 Combinación de métodos

Ningún método encuentra todo. La revisión del código atrapa errores temprano pero no muestra lo que pasa al ejecutar; las pruebas con la aplicación funcionando muestran el comportamiento real pero llegan más tarde; lo automático es rápido pero superficial; lo manual es profundo pero lento. Por eso conviene combinarlos y repetir el proceso (Figura 4), en lugar de depender de uno solo.

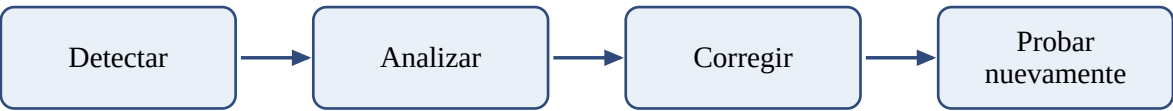


Figura 4. Ciclo de protección: se repite en cada versión.

7. RESULTADOS Y ANÁLISIS

7.1 Principales resultados

De acuerdo con la información verificada del artículo, la revisión se basó en 53 estudios y concluye que no hay consenso sobre cuál es la mejor metodología de evaluación de seguridad ni criterios estandarizados para elegirla.

7.2 Vulnerabilidades más importantes

Las palabras clave del artículo destacan la inyección SQL y el Cross-Site Scripting como las vulnerabilidades centrales del análisis, lo que coincide con su posición en las listas de riesgos más comunes de OWASP.

7.3 Comparación de métodos

El artículo compara metodologías y herramientas de detección. Un fragmento del resumen indica que se analizaron técnicas y herramientas (como OWASP ZAP, Burp Suite y Nikto) según sus fortalezas y debilidades.

7.4 Ventajas

Un punto fuerte del trabajo es que reúne muchos estudios en uno solo, lo que da una visión más general que una prueba aislada; además su conclusión es honesta al reconocer la falta de consenso. Para un equipo de desarrollo, esto es útil porque muestra que combinar métodos es más sensato que apostar por uno.

Otra ventaja es que sus conclusiones son prácticas: orientan a los profesionales a no confiar ciegamente en una sola herramienta y a documentar los criterios con que escogieron sus métodos.

7.5 Limitaciones

Al ser una revisión de estudios previos, depende de la calidad de esos estudios.

Las herramientas y amenazas cambian rápido, por lo que las comparaciones envejecen.

Al no existir criterios comunes, es difícil comparar resultados entre estudios.

7.6 Nuestra opinión

Nos parece valioso que el artículo no venda una solución mágica y admita que falta estandarización. Como estudiantes, vemos que esa falta de criterios se refleja en la práctica: muchos proyectos escogen una herramienta por costumbre y no por comparación.

También creemos que el tema es muy oportuno: la cantidad de aplicaciones web crece más rápido que la cantidad de personas capacitadas en seguridad, y las revisiones sistemáticas ayudan a ordenar el conocimiento disponible.

7.7 ¿Estamos de acuerdo?

Sí, estamos de acuerdo con la conclusión principal. Coincide con lo que se recomienda en la práctica: combinar revisión de código, pruebas con la aplicación funcionando, herramientas automáticas y revisión manual. Estar de acuerdo no impide notar que el artículo sería más útil para desarrolladores principiantes si incluyera una guía concreta de cuándo usar cada método.

8. APLICACIÓN PRÁCTICA

8.1 Mejoras propuestas

Incluir una tabla simple que indique qué metodología conviene según el tamaño y la etapa del proyecto.

Agregar ejemplos concretos de aplicación en proyectos pequeños o académicos.

Incluir vulnerabilidades de control de acceso y autenticación con más peso, además de la inyección y el XSS.

8.2 Medidas de seguridad

Consultas parametrizadas y validación de todos los datos de entrada.

Escapar los datos al mostrarlos en las páginas.

Tokens antiCSRF y cookies de sesión con SameSite, HttpOnly y Secure.

Contraseñas con hash y límite de intentos de inicio de sesión.

Verificar roles en el servidor para cada ruta sensible.

Revisiones periódicas con herramientas automáticas y revisión manual.

Estas medidas siguen la lógica del artículo: no se trata de una sola solución, sino de varias capas que se complementan.

8.3 Aplicación de lo aprendido

Aplicaremos lo aprendido en tres pasos: revisar el código con una lista de control basada en las vulnerabilidades del capítulo 5; probar la aplicación en funcionamiento en un ambiente local; y repetir el ciclo cada vez que se agreguen funciones nuevas.

8.4 Proyecto de Cátedra

8.4.1 Nombre

Café Don Bosco: sistema web para una cafetería con una tienda en línea (los clientes pueden pedir sin crear cuenta) y un panel de administración para catálogo, inventario, ventas e historial. Está desarrollado en Java 17 con Jakarta EE (Servlets, JSP y JDBC), MySQL 8 y Apache Tomcat 10.1.

8.4.2 Problemas de seguridad

Los riesgos que más se aplican a un sistema como este son:

Inyección SQL, porque todo el acceso a datos se hace con JDBC escrito a mano.

XSS, porque los nombres de productos, notas del pedido y datos de clientes se muestran en páginas JSP.

CSRF, porque el panel de administración y el carrito usan cookie de sesión.

Autenticación: ataques de fuerza bruta contra el inicio de sesión del administrador.

Autorización: que un cliente llegue a pantallas de administración o vea el ticket de otro cliente.

8.4.3 Medidas

Revisando el código del repositorio, encontramos que el proyecto ya aplica varias de las medidas estudiadas:

Los DAO usan PreparedStatement con parámetros (por ejemplo, las consultas por id y por token_ticket), y las partes concatenadas son solo listas fijas de columnas.

Las vistas JSP escapan los datos con fn:escapeXml antes de mostrarlos.

El archivo META-INF/context.xml configura la cookie de sesión con SameSite estricto, como defensa contra CSRF.

Existe un CorsFilter que evita que otros sitios hagan peticiones autenticadas con la cookie de la víctima.

Las contraseñas se manejan con jBCrypt (PasswordUtil) y hay un LoginRateLimiter para limitar los intentos.

RolAdminFilter y SesionVistaFilter restringen el acceso a las rutas de administración según la sesión y el rol.

El ticket de la tienda se consulta por un token_ticket y no por un número consecutivo, lo que dificulta ver tickets ajenos.

Hay una bitácora (BitacoraDAO) y un GlobalExceptionHandler para no mostrar errores técnicos al usuario.

Como mejoras pendientes proponemos: agregar un token antiCSRF explícito en los formularios (hoy la defensa se apoya en SameSite), añadir encabezados como Content-Security-Policy, y ampliar las pruebas automáticas de seguridad.

8.4.4 Revisión del sistema

Combinaremos los métodos del capítulo 6: (1) revisión del código buscando concatenaciones en SQL y datos sin escapar en los JSP; (2) pruebas en un ambiente local con OWASP ZAP contra la tienda y el panel de administración; (3) revisión manual de los permisos, probando entrar como cliente a las rutas de administración y como un cliente a recursos de otro cliente; (4) corregir y probar nuevamente.

8.4.5 Beneficios

Esperamos reducir la probabilidad de filtrar datos de clientes, proteger la integridad de las ventas y del inventario, ganar confianza de los usuarios y dejar un procedimiento de revisión que pueda repetirse en futuras versiones.

9. CONCLUSIÓN PERSONAL

9.1 Qué aprendimos

Aprendimos que una revisión sistemática es una forma ordenada de resumir lo que muchos investigadores han encontrado, y que su valor está tanto en sus resultados como en reconocer lo que todavía falta resolver.

9.2 Qué aprendimos sobre seguridad

Que la seguridad no es una función que se agrega al final, sino una forma de trabajar durante todo el desarrollo, y que proteger una aplicación exige varias capas.

9.3 Qué aprendimos sobre vulnerabilidades

Que la mayoría nace de un mismo error: confiar en los datos que llegan del usuario o confiar en que la sesión basta para autorizar cualquier acción. Con prevenciones sencillas, como consultas parametrizadas y escape de datos, se evita una gran parte de los ataques comunes.

9.4 Qué aprendimos sobre detección

Que no hay un método único: lo automático y lo manual, y el código y la aplicación en ejecución, se complementan. Lo más importante es repetir las revisiones.

9.5 Cambio de perspectiva

Antes pensábamos en probar que el sistema funcione; ahora también pensamos en probar que no se pueda abusar de él. Ahora revisamos nuestro propio código como lo haría un atacante.

También entendimos que revisar la seguridad no es una tarea que se hace una vez: cada función nueva puede abrir un nuevo problema, por lo que la revisión debe ser parte del ciclo normal de desarrollo.

9.6 Reflexión final

Construir software implica la responsabilidad de proteger a quienes lo usan. Este trabajo nos dio un vocabulario y un método para asumirla, y el reto ahora es aplicarlo en cada proyecto.

10. REFERENCIAS BIBLIOGRÁFICAS

10.1 Artículo principal

Quesquen Farroñay, J. A., Bances Quevedo, J. J., Garces Rosendo, E. J., & Osoreo Granda, O. E. (2025). Evaluation of methodologies in web application security: A systematic literature review. *Proceedings of the LACCEI International Multi-Conference for Engineering, Education and Technology*. LACCEI. <https://doi.org/10.18687/LACCEI2025.1.1.526>

10.2 Otras fuentes

Grassi, P. A., Fenton, J. L., Newton, E. M., Perlner, R. A., Regenscheid, A. R., Burr, W. E., Richer, J. P., Lefkovitz, N. B., Danker, J. M., Choong, Y.-Y., Greene, K. K., & Theofanos, M. F. (2017). *Digital identity guidelines: Authentication and lifecycle management* (NIST Special Publication 800-63B). National Institute of Standards and Technology. <https://doi.org/10.6028/NIST.SP.800-63b>

OWASP Foundation. (2021). *OWASP Top 10:2021*. <https://owasp.org/Top10/>

10.3 Documentación

OWASP Foundation. (s. f.). *SQL Injection Prevention Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.). *Cross Site Scripting Prevention Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (s. f.). *Cross-Site Request Forgery Prevention Cheat Sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Cafe Don Bosco. (s. f.). *Código fuente y documentación del proyecto* [Repositorio]. Proyecto de Cátedra, Universidad Don Bosco.

11. ANEXOS

11.1 Diagramas

Los diagramas se encuentran en las figuras 1 a 4 del documento (capítulos 4 y 6).

11.2 Tablas

La tabla de comparación de métodos está en la sección 6.5. A continuación, un resumen de las vulnerabilidades estudiadas.

Vulnerabilidad	Causa principal	Prevención clave
Inyección SQL	Consultas armadas con texto del usuario	Consultas parametrizadas
XSS	Datos mostrados sin escapar	Escape de salida
CSRF	Peticiones aceptadas solo por la cookie	Token antiCSRF y SameSite
Autenticación	Contraseñas y sesiones débiles	Hash, límite de intentos, sesiones seguras

Vulnerabilidad	Causa principal	Prevención clave
Autorización	Permisos no verificados en el servidor	Verificar rol y propietario siempre

Tabla 2. Resumen de vulnerabilidades.

11.3 Material adicional

Lista de control para revisar el código: (a) ¿toda consulta SQL usa parámetros?; (b) ¿todo dato mostrado se escapa?; (c) ¿los formularios que cambian datos están protegidos contra CSRF?; (d) ¿las contraseñas se guardan con hash y hay límite de intentos?; (e) ¿cada ruta sensible verifica el rol en el servidor?